SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 3 – CO3

Course Outcome: CO3 – Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

CO3 Coverage: *CLASSIFICATION AND PREDICTION: Issues regarding classification and prediction, decision tree induction, Bayesian classification, rule-based classification, backpropagation, support vector machines, associative classification, accuracy and error measures, evaluating a classifier or predictor (cross-validation and bootstrap), metrics for performance evaluation, prediction algorithms, and the Weka tool for mining.*

Mode: Pair Work – **2 Students per Group** | **Duration:** 1 Week | **Total Marks:** 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Calculate → Implement → Analyse → Evaluate

Q1 – Decision Tree Induction: Choosing the Splitting Attribute — 6 Marks

A bank records the following loan decisions:

#	Income	Credit History	Employment	Approved
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No
8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Shared Group Work

Together:

- Identify the class attribute, the candidate splitting attributes and the stopping conditions
- Agree on the splitting criterion to be used for the manual tree and on the pruning approach

- Install Weka and load the same data (ARFF/CSV) for the experiments

Student A – Manual Induction

Student A must individually:

- Compute the **entropy** of the full data set and the **information gain** of every attribute at the root
- Compute the **gain ratio** or the **Gini index** for any one attribute and state whether it changes the choice of the root
- Build the complete tree and extract it as a set of paths
- Explain what happens if an attribute has many distinct values (for example a customer ID)

Student B – Weka Validation

Student B must individually:

- Run **J48** on the 12 tuples, and then on the *German Credit (credit-g)* data set that ships with Weka
- Compare Student A's manual tree with the Weka tree and explain every difference
- Vary the pruning confidence factor and minimum instances per leaf; report tree size and accuracy
- Explain **overfitting** using your own evidence

Individual Verification

Each student must be able to explain the other student's part of Q1.

Evidence Expected

Manual gain computations, the tree, the Weka output with parameter settings, and an interpreted comparison.

Q2 – Bayesian Classification and the Cost of Assumptions 6 Marks

Shared Group Work

Using the loan data of Q1, classify the unseen applicant **X = (Income = Low, Credit History = Good, Employment = Self-employed)** using the Naïve Bayes classifier.

Student A – Calculation

- Compute the prior probabilities and all required conditional probabilities
- Find the posterior (proportional) value for each class and the predicted class
- Show what happens if one conditional probability is **zero**, and fix it with **Laplacian correction**; recompute

Student B – Investigation

- Run **NaiveBayes** in Weka on the same data and verify Student A's probabilities
- Create a data set (real or synthetic) in which two attributes are **strongly correlated**, and measure how the accuracy of Naïve Bayes changes compared with a decision tree
- Explain why Naïve Bayes can still give useful class rankings although the independence assumption is violated

Joint Conclusion

> **When would you prefer Naïve Bayes over a decision tree, and when would you not?**

Evidence Expected

Probability tables, posterior calculations, Laplace correction, Weka verification and evidence for the independence-assumption study.

Q3 – Rule-Based and Associative Classification 7 Marks

Shared Group Work

Together, convert the Q1 decision tree into **IF–THEN rules** and discuss how rules represent knowledge differently from a tree.

Student A – Rule Quality

- Extract the rules and compute **coverage** and **accuracy** of each rule on the 12 tuples
- Explain rule **conflict resolution** (size ordering, rule ordering) with one example from the data
- Explain the sequential-covering idea for learning rules one at a time, and what is removed after each rule is learned

Student B – Associative Classification

- Run **JRip** (or **PART**) in Weka on *credit-g* and compare the rules with those obtained from J48
- Explain how **associative classification** (class-association rules) differs from the rules above, and what the roles of support and confidence are
- Mine class-association rules from the 12-tuple data with minimum support 25% and confidence 70%, and list those predicting *Approved* = *Yes*

Joint Design Question

> **A loan officer must be able to explain every rejection to the customer. Which of the three representations (tree, rules, associative rules) would you recommend, and why?**

Evidence Expected

Rule list with coverage/accuracy, JRip/PART output, class-association rules and a justified recommendation.

Q4 – Backpropagation: What Does the Network Learn? 7 Marks

Shared Group Work

Consider a network with 2 inputs, 2 hidden units and 1 output unit using the sigmoid activation. Inputs x1 = 1, x2 = 0, target t = 1, learning rate 0.9. Initial weights: w(x1→h1) = 0.2, w(x2→h1) = 0.4, w(x1→h2) = −0.3, w(x2→h2) = 0.1, w(h1→o) = −0.3, w(h2→o) = −0.2; biases: h1 = −0.4, h2 = 0.2, o = 0.1.

Student A – One Training Step

- Compute the forward pass: output of each hidden unit and the output unit
- Compute the error terms of the output and hidden units
- Update all weights and biases for one iteration and recompute the output; did the error decrease?
- Explain why the weights are initialised to small random values and what happens if all are set to the same value

Student B – Behaviour of the Learner

- Train a **MultilayerPerceptron** in Weka (or equivalent in Python) on a real data set
- Vary the learning rate, the number of hidden units and the number of epochs; record training and test accuracy
- Identify and demonstrate **overfitting** and **slow/unstable training**
- Compare training time and interpretability with the decision tree of Q1

Individual Verification

Each student must be able to explain how the output error propagates back to a hidden-layer weight.

Evidence Expected

Hand calculations for one iteration, tables of experiments and a discussion of strengths and limitations of neural classifiers.

Q5 – Support Vector Machines: Margin, Kernels and Scaling 7 Marks

Shared Group Work

Six points in 2-D: Class +1 → (3,3), (4,3), (4,4); Class −1 → (1,1), (1,2), (2,1). Together, plot them and discuss what a **maximum-margin** separator means.

Student A – Geometry

- Identify by inspection the **support vectors**, and write a separating hyperplane $w \cdot x + b = 0$ that is equidistant from the two classes
- Compute the margin width for your hyperplane
- Explain what changes if a non-support-vector point is removed, and what changes if a support vector is removed
- Explain the role of the **soft margin parameter C** and of **kernels** for non-linearly separable data

Student B – Experiments

- Using **SMO** in Weka (or SVC in Python), classify a non-linear data set (for example concentric circles or a real data set) with linear, polynomial and RBF kernels
- Study the effect of C and kernel parameters on accuracy and the number of support vectors
- Show the effect of **not scaling the attributes** and then of scaling them

Joint Conclusion

> **For which type of data would you choose an SVM, and what practical limitations did your experiments reveal?**

Evidence Expected

Plot with support vectors, hyperplane and margin, kernel/parameter experiments and the effect of scaling.

Q6 – Evaluating Classifiers and Predictors — 7 Marks

Shared Group Work

A fraud-detection classifier is tested on 1,000 transactions and produces: **TP = 40, FN = 20, FP = 30, TN = 910**. Together, discuss which measure is meaningful for this problem.

Student A – Metrics and Prediction Error

- Compute accuracy, error rate, **sensitivity (recall), specificity, precision and F-measure**
- Show that a classifier that predicts “not fraud” for every transaction obtains a high accuracy, and explain the **class-imbalance problem**
- For actual values {12, 15, 20, 25} and predicted values {10, 18, 19, 30}, compute **MAE, MSE, RMSE** of a predictor and explain when RMSE is preferred over MAE
- Explain how a **ROC curve** is constructed and how to read it

Student B – Estimation Methods

- On an imbalanced data set (for example *credit-g* or a fraud data set), compare **holdout, 10-fold cross-validation and bootstrap (.632)** estimates of accuracy
- Repeat holdout with at least five random splits and report the variation in accuracy
- Compare J48, Naïve Bayes and SMO using paired evaluation (for example Weka Experimenter) and report whether the differences are meaningful
- Plot a ROC curve for one classifier

Individual Verification

Each student must be able to explain why accuracy alone is misleading and which estimation method they would trust and why.

Evidence Expected

Metric calculations, error measures, estimation-method comparison with variation and an interpreted classifier comparison.

DAY-4 DESIGN CHALLENGE

Q7 – Design a Classification Solution from First Principles 10 Marks

Problem: A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% of patients are readmitted. Missing a high-risk patient (false negative) is judged **ten times more costly** than flagging a patient who is not at risk (false positive). Doctors must be able to understand the reason for each alert. Data: age, diagnosis group, length of stay, number of prior admissions, lab results and discharge medications (some values missing).

The Day-4 problem must be solved **collaboratively**. Neither student may work on this question independently and simply combine the answers later.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other, **Student A** and **Student B** each independently propose:

- The classification technique(s) they would use
- Preprocessing and handling of imbalance and missing values
- The evaluation measure and estimation method
- How the model's decisions will be explained to doctors

Each student's initial proposal must be retained in the submission. **No marks are deducted merely because an initial design is incorrect.** The purpose is to provide evidence of **independent thinking**.

Stage 2 – Compare the Two Designs

Discuss where the designs agree and differ, the strengths and weaknesses of each, missing cases, and which ideas should be retained. Record this comparison.

Stage 3 – Joint Final Design

Develop the final solution, identifying the chosen technique, preprocessing, parameters, evaluation method and cost-sensitive decision rule.

Stage 4 – Divide the Validation Responsibilities

Student A – Analytical Validation

- Compute the expected cost of two alternative decision thresholds using a confusion matrix and the 10:1 cost ratio
- Explain why accuracy is an inappropriate selection measure here
- Explain the strengths and limitations of the chosen technique for this data

Student B – Experimental Validation

- Implement the final design and a simple baseline in Weka/Python on a public analogue data set (for example a diabetes-readmission or imbalanced medical data set)
- Evaluate with cross-validation using recall, precision and ROC/AUC, and total cost
- Show the effect of the imbalance-handling method (for example resampling or cost-sensitive learning)

Stage 5 – Cross-Defence

Student A must explain:

> **What experimental evidence supports the group's design?**

Student B must explain:

> **How were the cost ratio and the decision threshold derived from the confusion matrix?**

Stage 6 – Joint Evaluation

Together determine why the problem is difficult for standard accuracy-driven classifiers, what you sacrificed to obtain interpretable alerts, and under what conditions the design would fail. Conclude:

> **What characteristics of a data set and a business requirement should a data miner examine before choosing a classifier?**

Evidence Expected
Both independent proposals, a design-comparison record, the final design, analytical and experimental validation, and a joint evaluation.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Decision Tree Induction	**6**
Q2	Bayesian Classification	**6**
Q3	Rule-Based and Associative Classification	**7**
Q4	Backpropagation	**7**
Q5	Support Vector Machines	**7**
Q6	Evaluating Classifiers and Predictors	**7**
Q7	**Day-4 Design Challenge – Classification Solution**	**10**
TOTAL		**50**

CONTRIBUTION RECORD

Each pair must include the following table at the end of the submission.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

The contribution record must reflect the actual work completed. **Primary contribution does not imply exclusive responsibility. Both students remain accountable for the complete assignment.**

INDIVIDUAL VERIFICATION

Faculty may select **any question**, including a component for which the student was not designated as the primary contributor. Each student must be able to:

- Explain the calculations and the tool settings used
- Interpret the evaluation measures and experimental results
- Justify design decisions
- Respond to a modified data set or requirement

A student should not answer:

> **"My partner did this section."**

NOTE: RECOMMENDED FACULTY VERIFICATION MODEL (VIVA)

For each student, conduct a brief **4–5 minute verification**. Ask:

Question 1 – Primary responsibility: one technical question from the component for which the student had primary responsibility.

Question 2 – Partner's responsibility: one question from a component primarily completed by the partner.

Question 3 – Transfer question: modify a condition and ask the student to reason about the consequence. For example:

> **“If the readmission rate falls from 6% to 0.5%, what happens to your precision, and would you change the decision threshold?”**
>
> **“If Income were replaced by a customer ID attribute with a unique value for each tuple, what would the decision tree do and why?”**
>
> **“If the SVM attributes are not scaled and one attribute is measured in thousands, what happens to the margin?”**

Q1 – DECISION TREE INDUCTION: CHOOSING THE SPLITTING ATTRIBUTE (6 MARKS)

Class attribute: Approved (Yes/No). Candidate splitting attributes: Income, Credit History and Employment. The 12 training tuples contain 5 Yes and 7 No decisions. For the manual tree, use ID3 information gain. Stop when a node is pure, no attributes remain, or no positive information gain is available. If a branch has no training examples, use the parent node's majority class. For pruning, compare the unpruned tree with reduced-error or J48 confidence-factor pruning on validation data.

1.1 Entropy of the full dataset

Entropy(S) = $-p(\text{Yes})\log_2 p(\text{Yes}) - p(\text{No})\log_2 p(\text{No})$. Here p(Yes)=5/12 and p(No)=7/12.

```
H(S) = -(5/12)log2(5/12) - (7/12)log2(7/12) = 0.9799 bits
```

1.2 Information gain at the root

Attribute / value	Yes	No	Subset entropy
Income = High	3	1	0.8113
Income = Medium	1	3	0.8113
Income = Low	1	3	0.8113
Credit = Good	4	2	0.9183
Credit = Poor	1	5	0.6500
Employment = Salaried	3	3	1.0000
Employment = Self-employed	2	4	0.9183

For Income, each value has 4 tuples: remainder = (4/12)(0.8113)+(4/12)(0.8113)+(4/12)(0.8113)=0.8113.
Gain(Income)=0.9799−0.8113=0.1686.

For Credit History, each value has 6 tuples: remainder=(6/12)(0.9183)+(6/12)(0.6500)=0.7842.
Gain(Credit)=0.9799−0.7842=0.1957.

For Employment, each value has 6 tuples: remainder=(6/12)(1.0000)+(6/12)(0.9183)=0.9591.
Gain(Employment)=0.9799−0.9591=0.0207.

Root attribute	Weighted entropy after split	Information gain
Income	0.8113	0.1686
Credit History	0.7842	0.1957
Employment	0.9591	0.0207

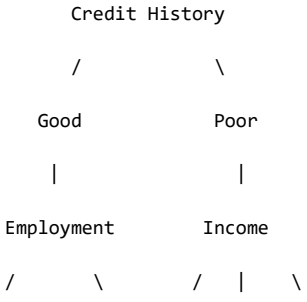
Credit History has the largest information gain (0.1957), so it is selected as the root.

1.3 Gain ratio and Gini check

Credit has two equally sized branches, so SplitInfo(Credit)=$-0.5\log_2 0.5 - 0.5\log_2 0.5$=1 bit. GainRatio(Credit)=0.1957/1=0.1957. The gain ratio therefore does not change the root choice.

Gini(S)=$1-(5/12)^2-(7/12)^2$=0.4861. For Credit=Good, Gini=$1-(4/6)^2-(2/6)^2$=0.4444; for Credit=Poor, Gini=$1-(1/6)^2-(5/6)^2$=0.2778. Weighted Gini=0.3611; Gini reduction=0.1250. This is larger than the Gini reductions for Income (0.1111) and Employment (0.0139), so Gini also selects Credit History.

1.4 Complete manual tree



```
Salaried  Self-employed High Medium Low

   Yes        Income       Employment/majority

              / | \       High: Salaried→No, Self-employed→Yes

           High Med Low    Medium→No; Low→No

            Yes  No  No
```

For Credit=Poor, the High branch is split by Employment: Salaried is No (tuple 2), Self-employed is Yes (tuple 12). Medium and Low are pure No. For Credit=Good, Salaried is pure Yes; Self-employed is split by Income: High is Yes, Medium and Low are No.

1.5 Tree paths, high-cardinality attributes and stopping

IF conditions	THEN Approved	Training coverage	Accuracy
Credit=Good AND Employment=Salaried	Yes	3	3/3 = 100%
Credit=Good AND Employment=Self-employed AND Income=High	Yes	1	1/1 = 100%
Credit=Good AND Employment=Self-employed AND Income∈{Medium,Low}	No	2	2/2 = 100%
Credit=Poor AND Income=High AND Employment=Salaried	No	1	1/1 = 100%
Credit=Poor AND Income=High AND Employment=Self-employed	Yes	1	1/1 = 100%
Credit=Poor AND Income∈{Medium,Low}	No	4	4/4 = 100%

A customer ID with nearly one unique value per tuple can create many tiny branches and may perfectly memorise training labels. This produces a large tree with poor generalisation. Avoid identifier attributes as predictors unless they have a meaningful, validated relationship with the target. Use minimum leaf size, pruning, validation and domain review.

1.6 Weka validation – required experiment

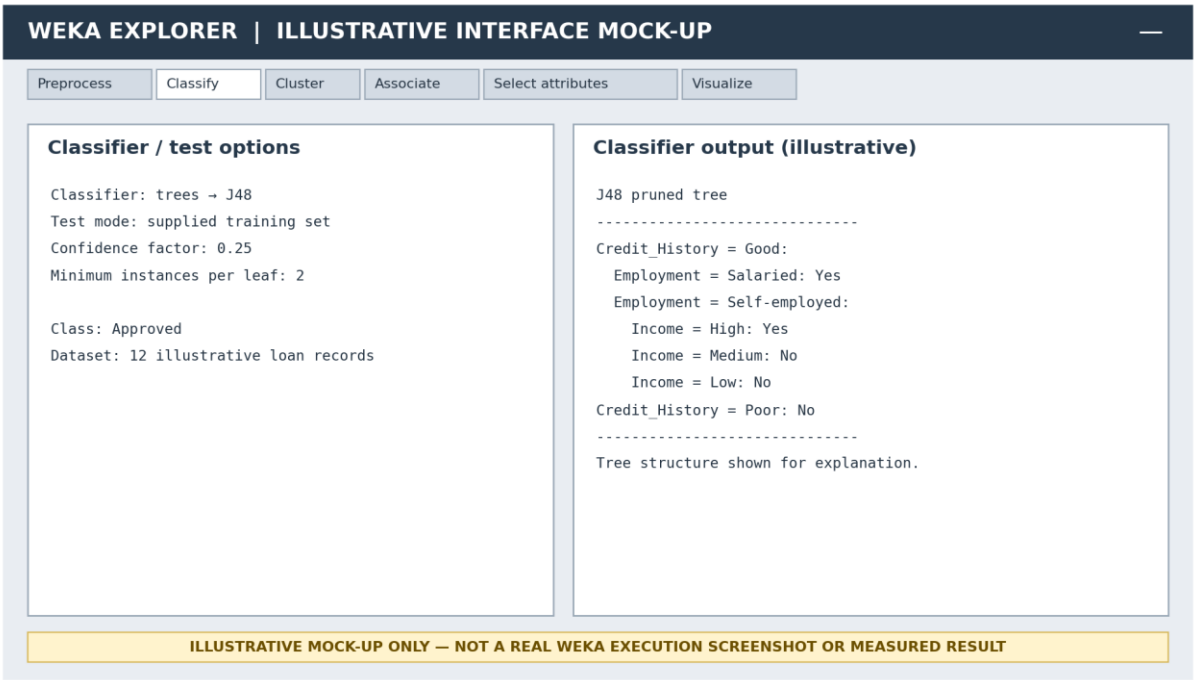


Figure 1. Illustrative Weka Explorer layout for the J48 decision-tree experiment.

In Weka Explorer, create a CSV/ARFF with the 12 tuples and set Approved as the class. Choose J48. Record confidence factor (C) and minimum number of instances per leaf (M). Run first with defaults, then vary C (for example 0.25, 0.10, 0.05) and M (1, 2, 3). Record number of leaves, tree size and accuracy. Then load the Weka credit-g dataset and repeat. The J48 tree may differ from the manual ID3 tree because J48 uses gain ratio, pruning, minimum-leaf constraints, missing-value handling and implementation-specific tie breaking. Add screenshots and actual values from your installation.

Dataset / settings	Tree size / leaves	Accuracy	Observations
12 tuples – default J48	Fill after run	Fill after run	Compare root and leaves with manual tree.
12 tuples – C=0.10, M=2	Fill after run	Fill after run	More pruning / larger leaf constraint may simplify tree.
credit-g – default J48	Fill after run	Fill after run	Record Weka test mode and output.
credit-g – varied C and M	Fill after run	Fill after run	Use same evaluation method for fair comparison.

Q2 – BAYESIAN CLASSIFICATION AND THE COST OF ASSUMPTIONS (6 MARKS)

Unseen applicant X=(Income=Low, Credit=Good, Employment=Self-employed). Naïve Bayes assumes predictors are conditionally independent given the class. The posterior is proportional to prior × product of conditional probabilities.

2.1 Priors and conditional probabilities

There are 5 Yes and 7 No examples: P(Yes)=5/12=0.4167; P(No)=7/12=0.5833.

Probability	Yes (5 rows)	No (7 rows)
P(Income=Low | class)	1/5 = 0.2000	2/7 = 0.2857
P(Credit=Good | class)	4/5 = 0.8000	2/7 = 0.2857
P(Employment=Self-employed | class)	2/5 = 0.4000	4/7 = 0.5714

2.2 Posterior calculation without smoothing

Yes score = (5/12)(1/5)(4/5)(2/5)=0.010667. No score=(7/12)(2/7)(2/7)(4/7)=0.015873. Normalised P(Yes|X)=0.010667/(0.010667+0.015873)=0.4018; P(No|X)=0.5982. Therefore predict Approved=No.

2.3 Zero probability and Laplace correction

No zero occurs in this particular applicant's required conditional counts, but the assignment asks us to show the zero-frequency problem. Suppose an unseen value such as Income=Very High has zero occurrences among Yes records. Then the unsmoothed product for Yes becomes zero, even if other evidence strongly supports Yes. Laplace (add-one) smoothing avoids this.

For a categorical attribute with k possible values, P(value|class)=(count+1)/(class_count+k). Income has 3 values; Credit has 2; Employment has 2.

Smoothed probability for X	Yes	No
P(Low | class)	(1+1)/(5+3)=0.2500	(2+1)/(7+3)=0.3000
P(Good | class)	(4+1)/(5+2)=0.7143	(2+1)/(7+2)=0.3333
P(Self-employed | class)	(2+1)/(5+2)=0.4286	(4+1)/(7+2)=0.5556

Smoothed Yes score=(5/12)(0.25)(0.7143)(0.4286)=0.03189. Smoothed No score=(7/12)(0.30)(0.3333)(0.5556)=0.03241. Normalised scores are approximately P(Yes|X)=0.4960 and P(No|X)=0.5040. Prediction remains No, but the result is now almost a tie. Minor rounding may change the last decimal.

2.4 Weka verification and correlated-attribute investigation

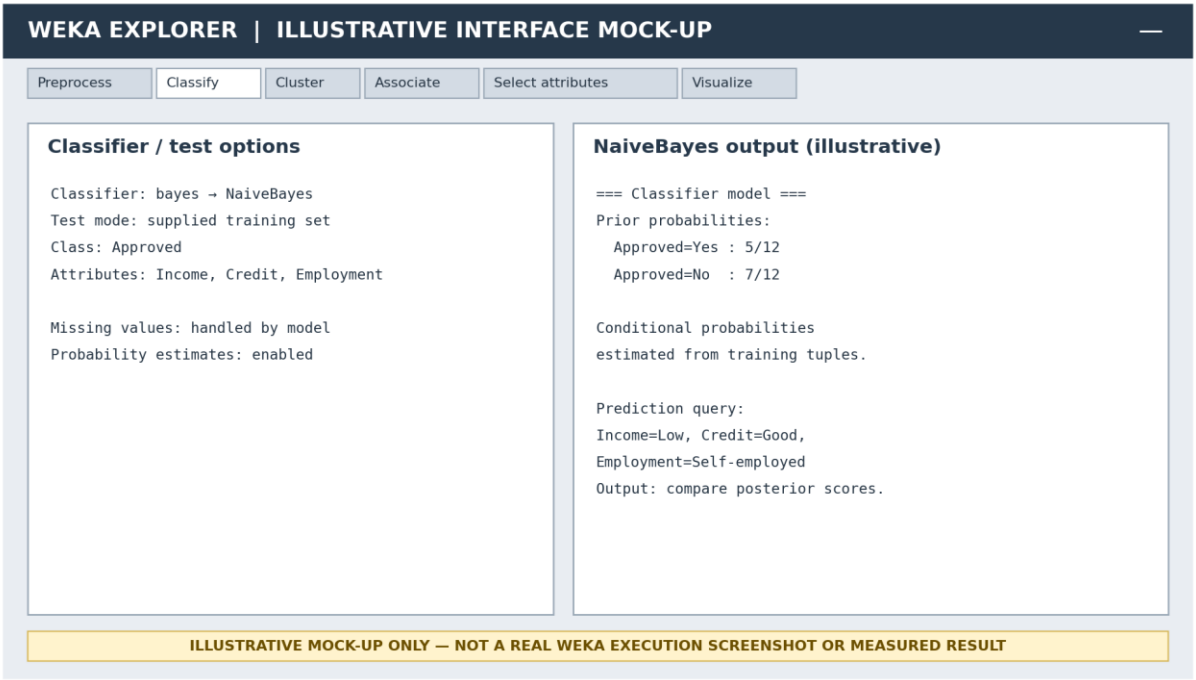


Figure 2. Illustrative Weka Explorer layout for Naïve Bayes classification.

For Weka verification, load the 12-row file, select NaiveBayes, and inspect the model output and predictions. Weka may use its own probability estimation and nominal handling; compare class distributions rather than expecting rounded values to be identical. For the investigation, create a reproducible dataset with two strongly correlated predictors (for example, monthly salary and annual salary=12×monthly salary) and a target generated from a noisy rule. Train NaiveBayes and J48 with identical stratified cross-validation folds. Record accuracy, precision, recall and confusion matrices. Do not claim a result until the experiment is run.

Naïve Bayes can still rank classes usefully when independence is violated because errors in conditional probability estimates may partly cancel in the posterior comparison. It is fast, handles high-dimensional sparse data well and often needs less training data. It can be less reliable when correlated predictors effectively count the same evidence multiple times.

2.5 Joint conclusion

Prefer Naïve Bayes when fast training, a small dataset, high-dimensional categorical features or probabilistic class ranking is important. Prefer a decision tree when transparent hierarchical rules and interactions are important. Do not assume either is universally superior: compare them using the same data split, suitable metrics and the cost of errors.

Q3 – RULE-BASED AND ASSOCIATIVE CLASSIFICATION (7 MARKS)

3.1 Rules extracted from the manual tree

Each root-to-leaf path becomes an IF–THEN rule. Coverage is the number of training tuples satisfying the rule. Rule accuracy is the fraction of covered tuples whose class matches the rule conclusion.

Rule	Coverage	Correct	Accuracy
R1: IF Credit=Good AND Employment=Salaried THEN Yes	3	3	100%
R2: IF Credit=Good AND Employment=Self-employed AND Income=High THEN Yes	1	1	100%
R3: IF Credit=Good AND Employment=Self-employed AND Income∈{Medium,Low}	2	2	100%

THEN No			
R4: IF Credit=Poor AND Income=High AND Employment=Salaried THEN No	1	1	100%
R5: IF Credit=Poor AND Income=High AND Employment=Self-employed THEN Yes	1	1	100%
R6: IF Credit=Poor AND Income∈{Medium,Low} THEN No	4	4	100%

The rules cover all 12 training tuples and are perfectly accurate on this small training set. This does not prove they will be 100% accurate on new applicants; the sample is tiny and the tree was constructed from these same records.

3.2 Conflict resolution and sequential covering

Conflicts can occur when two rules match the same applicant but predict different classes. Size ordering prefers the more specific rule (more conditions); rule ordering uses a priority list learned or arranged by confidence/accuracy/coverage. A default class handles applicants matching no rule. Example: a broad rule 'IF Credit=Poor THEN No' conflicts with the specific rule 'IF Credit=Poor AND Income=High AND Employment=Self-employed THEN Yes'. Size ordering chooses the specific Yes rule.

Sequential covering learns one rule at a time: find a rule that covers many positive examples and few negative examples, add it to the rule set, remove the positive examples already covered (or mark them as covered), and repeat until the stopping condition is met. Negative examples are not simply deleted from the whole training set; they remain important when evaluating future rules.

3.3 Associative classification

Associative classification mines class-association rules (CARs) of the form X → class, where X is a set of attribute-value items. Support is the fraction of all tuples containing both X and the class. Confidence is P(class|X), the fraction of tuples containing X that belong to that class. Minimum support 25% means at least 3 of 12 tuples; minimum confidence 70% means at least 70% of covered tuples must be in the predicted class.

CAR predicting Yes	Support count / 12	Confidence
Credit=Good AND Employment=Salaried → Yes	3/12 = 25.0%	3/3 = 100%
Income=High AND Credit=Good → Yes	2/12 = 16.7%	2/2 = 100% (below support; reject)
Income=High AND Employment=Self-employed → Yes	2/12 = 16.7%	2/2 = 100% (below support; reject)
Income=High → Yes	3/12 = 25.0%	3/4 = 75.0%
Credit=Good → Yes	4/12 = 33.3%	4/6 = 66.7% (below confidence; reject)

At the stated thresholds, the qualifying Yes rules include Credit=Good AND Employment=Salaried → Yes and Income=High → Yes. Other itemsets may also qualify if they meet both thresholds; use a frequent-itemset/CAR miner to enumerate all combinations and verify support/confidence. The examples above show the main qualifying patterns and near-misses.

3.4 JRip/PART in Weka and representation comparison

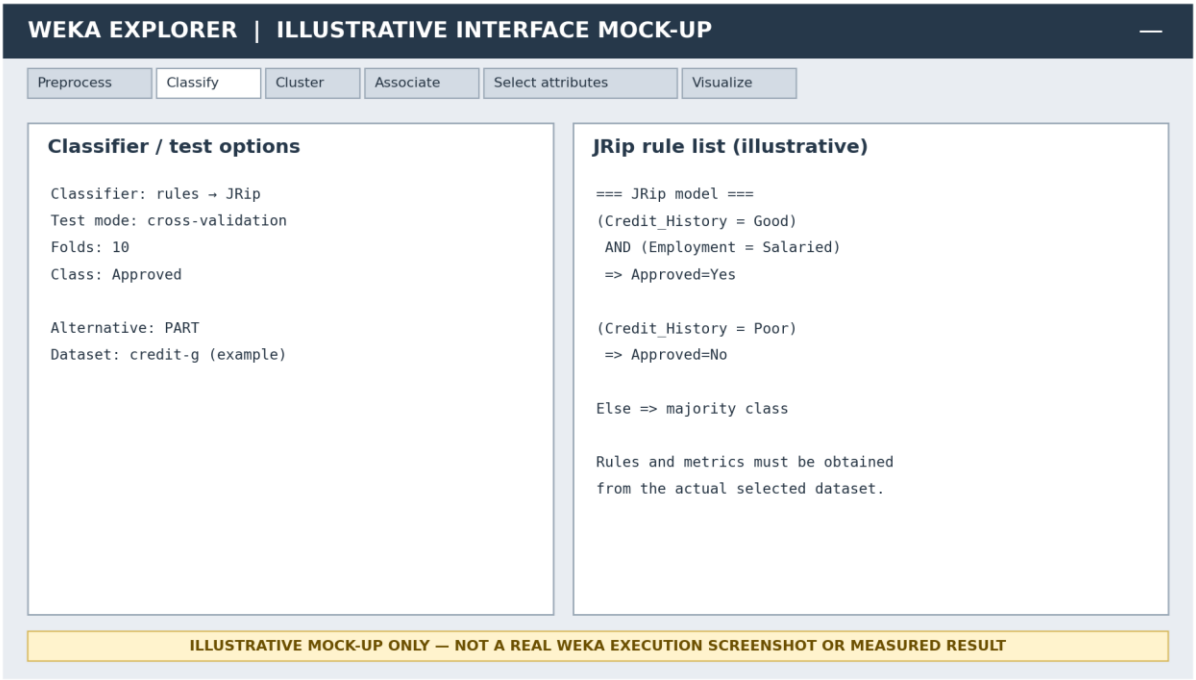


Figure 3. Illustrative Weka Explorer layout for JRip rule induction.

Run JRip (JRip is RIPPER-style rule induction) or PART on Weka's credit-g dataset. Record the generated rules, number of rules, test mode and evaluation metrics. Compare with J48: J48 expresses nested decisions as a tree; JRip/PART expresses ordered IF–THEN rules; associative classification mines frequent attribute combinations that predict a class and filters them using support/confidence. The latter may produce many overlapping rules.

For explaining a rejection, recommend a pruned decision tree or a short ordered rule list. Both can provide a path or conditions that a loan officer can communicate. Associative rules are useful for discovering common combinations but may overlap and do not necessarily provide one unambiguous explanation for each individual decision. Any real credit decision also needs fairness, legal and adverse-action review.

Q4 – BACKPROPAGATION: WHAT DOES THE NETWORK LEARN? (7 MARKS)

Network: two inputs, two hidden sigmoid units and one sigmoid output. x1=1, x2=0, target t=1, learning rate η=0.9. Initial weights and biases are exactly as stated in the assignment. Use online gradient descent and squared error $E=\frac{1}{2}(t-o)^2$.

4.1 Forward pass

Sigmoid σ(z)=1/(1+e^(−z)). Hidden unit h1 net=0.2(1)+0.4(0)−0.4=−0.2; h1=σ(−0.2)=0.450166. Hidden unit h2 net=−0.3(1)+0.1(0)+0.2=−0.1; h2=σ(−0.1)=0.475021.

Output net=−0.3(0.450166)−0.2(0.475021)+0.1=−0.130054. Output o=σ(−0.130054)=0.467532. Initial error $E=\frac{1}{2}(1-0.467532)^2=0.141761$.

4.2 Error terms (deltas)

For a sigmoid output with squared error, δo=(t−o)o(1−o)= (1−0.467532)(0.467532)(0.532468)=0.132556.

Hidden deltas: δh1=h1(1−h1)×w(h1→o)×δo = 0.450166(0.549834)(−0.3)(0.132556)=−0.009843. δh2=0.475021(0.524979)(−0.2)(0.132556)=−0.006611.

4.3 Weight and bias updates

For a weight into unit j, w_new=w_old+ηδj×input. For a bias, b_new=b_old+ηδj.

Parameter	Initial	Update	New

w(x1→h1)	0.200000	0.9×(−0.009843)×1	0.191141
w(x2→h1)	0.400000	0.9×(−0.009843)×0	0.400000
w(x1→h2)	−0.300000	0.9×(−0.006611)×1	−0.305950
w(x2→h2)	0.100000	0.9×(−0.006611)×0	0.100000
w(h1→o)	−0.300000	0.9×0.132556×0.450166	−0.246295
w(h2→o)	−0.200000	0.9×0.132556×0.475021	−0.143330
bias h1	−0.400000	0.9×(−0.009843)	−0.408859
bias h2	0.200000	0.9×(−0.006611)	0.194050
bias output	0.100000	0.9×0.132556	0.219300

4.4 Recomputed output and error

New h1=σ(0.191141−0.408859)=0.445785. New h2=σ(−0.305950+0.194050)=0.472054. New output net=(−0.246295)(0.445785)+(−0.143330)(0.472054)+0.219300≈0.04185; output=0.510460. New E=½(1−0.510460)²=0.119825. The error decreased from 0.141761 to 0.119825, so this update moved the network in the desired direction.

Small random initial weights break symmetry. If all hidden units start with identical weights and biases, they produce identical outputs and receive identical updates; they may remain identical and learn the same feature. Random initialization helps hidden units learn different representations.

4.5 Student B – Weka experiment record

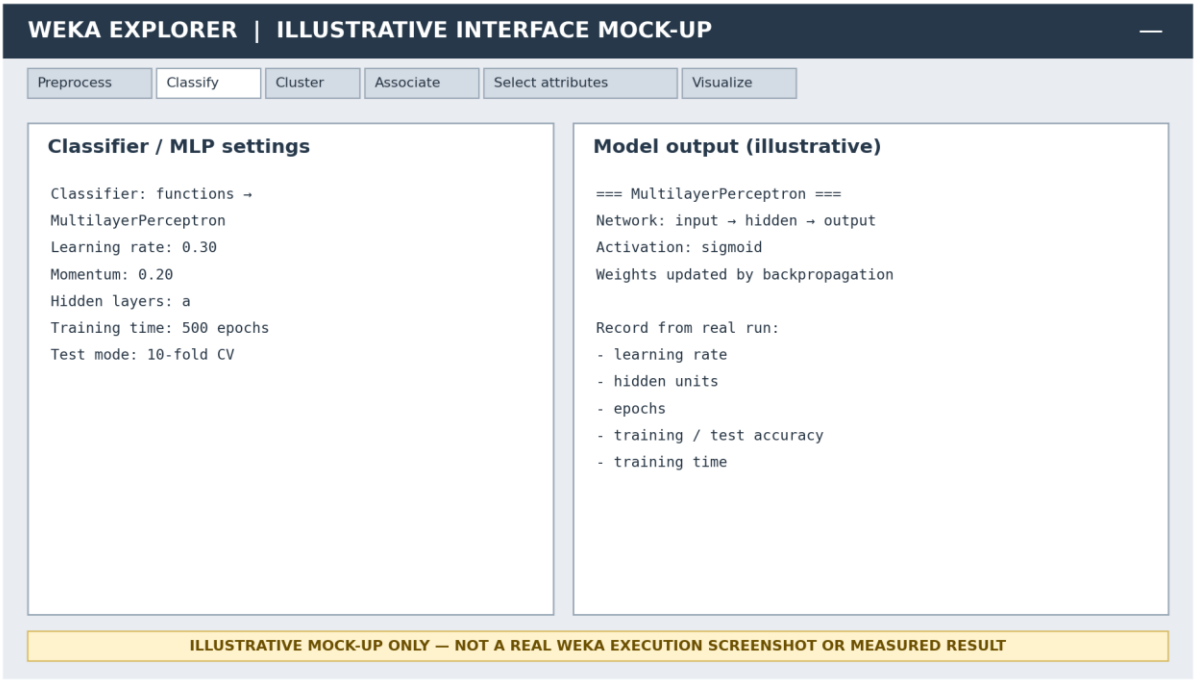


Figure 4. Illustrative Weka Explorer layout for MultilayerPerceptron.

Use Weka MultilayerPerceptron on a real dataset. Keep the same train/test split while varying learning rate, hidden units and epochs. Record training accuracy, test accuracy, training time and model settings. A large network or too many epochs can overfit: training accuracy rises while test accuracy stagnates or falls. Very high learning rates may cause unstable updates; very low rates may learn slowly.

Run	Learning rate	Hidden units	Epochs	Train accuracy	Test accuracy	Time
1	0.1	a (automatic)	500	Fill after run	Fill after run	Fill after run
2	0.3	a (automatic)	500	Fill after run	Fill after run	Fill after run
3	0.1	2	1000	Fill after run	Fill after run	Fill after run
4	0.1	10	1000	Fill after run	Fill after run	Fill after run
5	0.1	10	3000	Fill after run	Fill after run	Fill after run

Compared with a decision tree, a neural network can model complex nonlinear boundaries but generally needs more tuning and is less directly interpretable. Trees are usually faster to explain as rules, especially for small tabular datasets.

Q5 – SUPPORT VECTOR MACHINES: MARGIN, KERNELS AND SCALING (7 MARKS)

Positive points: (3,3), (4,3), (4,4). Negative points: (1,1), (1,2), (2,1). The positive triangle lies above/right of the negative triangle. A maximum-margin separator is the line that separates the convex hulls while maximizing the minimum perpendicular distance to either class.

5.1 Separating hyperplane and support vectors

The closest negative boundary is the segment joining (1,2) and (2,1), which lies on x1+x2=3. The closest positive point is (3,3), which lies on x1+x2=6. The midpoint separator is x1+x2=4.5. Therefore choose w=(1,1), b=−4.5, and decision boundary w·x+b=0.

Support vectors are (1,2) and (2,1) from the negative class and (3,3) from the positive class. They lie on the two margin boundaries x1+x2=3 and x1+x2=6. The other positive points are farther from the boundary.

5.2 Margin width

For w=(1,1), ||w||=√2. Distance from the separator to either support boundary is 1.5/√2=1.06066. Total margin width = 2/||w|| when expressed in canonical scaling; equivalently distance between parallel support lines = (6−3)/√2 = 3/√2 = 2.12132.

5.3 Removing points, soft margin and kernels

Removing a non-support-vector point normally does not change the optimal hyperplane because it does not determine the margin. Removing a support vector can change the margin or separator because it is one of the closest points defining the solution. The soft-margin parameter C controls the penalty for margin violations: large C penalizes training errors strongly and can produce a narrower, more complex boundary; small C permits more violations and stronger regularization. Kernels (linear, polynomial, RBF) compute similarity in an implicit feature space and allow nonlinear boundaries.

5.4 Student B – kernel and scaling experiment

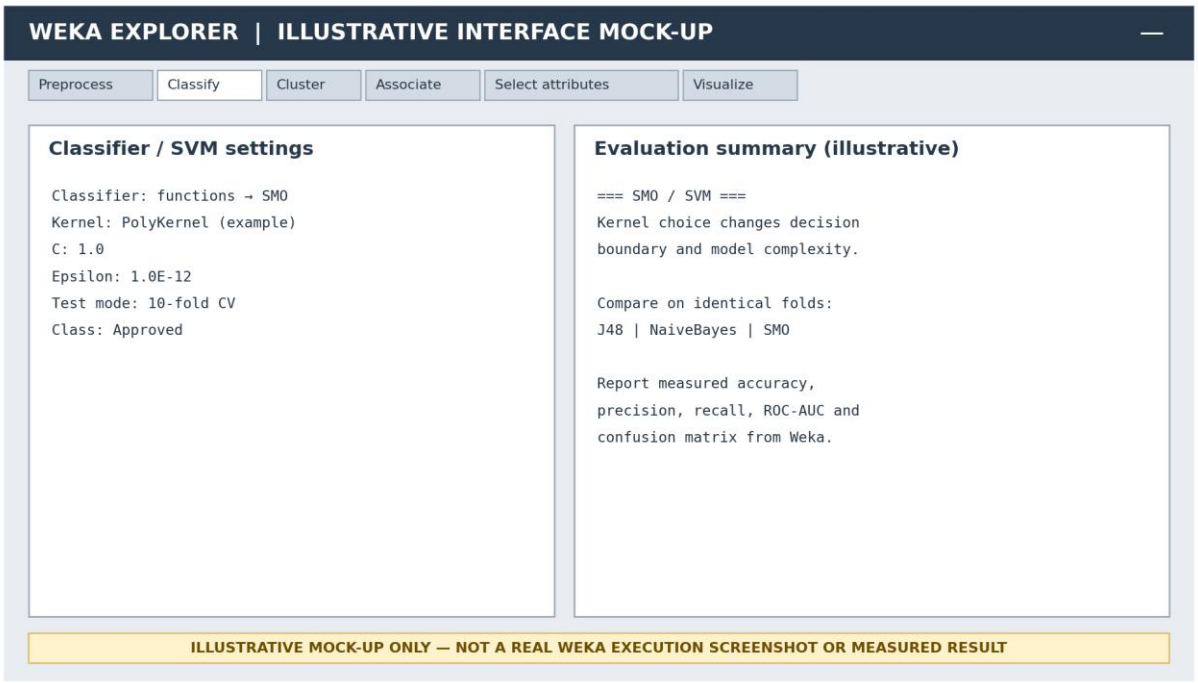


Figure 5. Illustrative Weka Explorer layout for SMO/SVM.

Use a nonlinear dataset such as concentric circles. In Python, generate it reproducibly with make_circles(random_state=42), split with stratification, and compare SVC(kernel='linear'), SVC(kernel='poly') and SVC(kernel='rbf'). Vary C and gamma; record accuracy, support-vector count and fit time. Then compare results with and without StandardScaler in a Pipeline. Scaling is important because SVM margins and kernel distances depend on feature magnitudes.

Kernel / setting	C	Gamma / degree	Scaled?	Accuracy	Support vectors
Linear	1	—	No	Run and record	Run and record
Linear	1	—	Yes	Run and record	Run and record
Polynomial	1	degree 3	Yes	Run and record	Run and record
RBF	1	scale	Yes	Run and record	Run and record
RBF	10	0.1	Yes	Run and record	Run and record
RBF	0.1	1.0	Yes	Run and record	Run and record

SVM is suitable for medium-sized, high-dimensional datasets with a meaningful margin and suitable kernel. Limitations include parameter tuning, scaling requirements, training cost on very large datasets and lower interpretability than a small decision tree.

Q6 – EVALUATING CLASSIFIERS AND PREDICTORS (7 MARKS)

6.1 Confusion matrix measures

Given TP=40, FN=20, FP=30, TN=910. Total=1000.

Measure	Formula	Calculation	Result
Accuracy	(TP+TN)/N	(40+910)/1000	0.9500 = 95.00%
Error rate	(FP+FN)/N	(30+20)/1000	0.0500 = 5.00%
Sensitivity / Recall	TP/(TP+FN)	40/60	0.6667 = 66.67%
Specificity	TN/(TN+FP)	910/940	0.9681 = 96.81%
Precision	TP/(TP+FP)	40/70	0.5714 = 57.14%
F1	2PR/(P+R)	2×0.5714×0.6667/(0.5714+0.6667)	0.6154 = 61.54%

The all-not-fraud classifier predicts every transaction as negative. It obtains TN=940 and FN=60, so accuracy=940/1000=94%, but recall for fraud=0. This illustrates why accuracy alone is not sufficient for imbalanced fraud detection.

6.2 Prediction errors

Actual={12,15,20,25}; predicted={10,18,19,30}. Errors actual−predicted={2,−3,1,−5}; absolute errors={2,3,1,5}; squared errors={4,9,1,25}.

```
MAE = (2+3+1+5)/4 = 11/4 = 2.75

MSE = (4+9+1+25)/4 = 39/4 = 9.75

RMSE = √9.75 = 3.1225
```

RMSE penalizes large errors more heavily because errors are squared before averaging. It is preferred when large prediction mistakes are especially costly and the squared-error scale is meaningful. MAE is more robust to outliers and easier to interpret as average absolute error.

6.3 ROC curve

A ROC curve plots True Positive Rate (sensitivity) on the y-axis against False Positive Rate (1−specificity) on the x-axis as the classification threshold changes. At a very high threshold, few cases are predicted positive; lowering the threshold generally increases both true positives and false positives. A curve closer to the top-left is better. AUC summarizes ranking performance across thresholds, but for very rare fraud, precision-recall curves and PR-AUC are often more informative.

6.4 Holdout, cross-validation and bootstrap comparison

Method	Procedure	Strength	Limitation
Holdout	One train/test split	Fast and simple	Result depends on the particular split.
Repeated holdout	Repeat at least five random stratified splits	Shows variation and sensitivity to split	Repeated test sets overlap; estimates are not independent.
10-fold cross-validation	Split into 10 folds; train on 9 and test on 1, repeat	Uses data efficiently and reduces single-split dependence	More computational cost; use stratified folds.
Bootstrap .632	Sample training sets with	Useful for small datasets and	Can be computationally

	replacement; evaluate out-of-bag cases; combine apparent and out-of-bag error using .632 weighting	estimator analysis	expensive and needs careful implementation.

Run J48, NaiveBayes and SMO on the same stratified folds using Weka Experimenter. Compare mean accuracy and standard deviation, plus class-specific precision/recall, ROC-AUC and PR-AUC where available. Paired testing is meaningful because each classifier is evaluated on the same folds. Do not claim a statistically significant difference from a small numerical gap alone.

Classifier	Accuracy mean ± SD	Fraud recall	PR-AUC	Notes
J48	Fill after run	Fill after run	Fill after run	Record pruning settings.
NaiveBayes	Fill after run	Fill after run	Fill after run	Record probability model.
SMO	Fill after run	Fill after run	Fill after run	Record kernel and scaling.

Q7 – DESIGN A CLASSIFICATION SOLUTION FROM FIRST PRINCIPLES (10 MARKS)

Problem: predict 30-day hospital readmission. Only 6% of patients are readmitted. A false negative costs 10 units and a false positive costs 1 unit. Doctors require understandable reasons. Available attributes: age, diagnosis group, length of stay, prior admissions, laboratory results and discharge medications, with some missing values.

7.1 Stage 1 – independent proposals (retain each student's original work)

Design element	Student A – initial proposal (complete independently)	Student B – initial proposal (complete independently)
Classifier		
Missing-value strategy		
Imbalance handling		
Evaluation metric / estimation		
Doctor-facing explanation		
Expected weakness		

The two initial proposals must be written separately before discussion, as required by the brief. The following is a suggested final design, not a replacement for those independent proposals.

7.2 Stage 2 and 3 – compare and final design

Suggested final classifier: a pruned decision tree (J48/CART) as the primary interpretable model, with logistic regression or a class-weighted ensemble as a benchmark. A shallow/pruned tree can express understandable alerts such as 'multiple prior admissions AND abnormal lab pattern AND long stay'. It may miss complex interactions, so performance must be validated.

Component	Final design
Prediction point	At discharge; only information available by discharge may be used.
Target	Readmitted within 30 days after discharge; define exclusions and follow-up completeness.
Missing data	Numeric labs: median imputation fitted on training folds plus missingness indicators; categorical diagnosis/medication: most-frequent or explicit Unknown category. Review clinically meaningful missingness.
Preprocessing	Check duplicates and impossible values; standardize codes; encode categories; scale only for models that require it.
Imbalance	Use class weights or cost-sensitive learning first; compare with training-fold resampling. Keep validation/test prevalence at the real 6%.
Model	Pruned decision tree for explainability; compare with cost-sensitive logistic regression / random forest as benchmark.
Decision threshold	Choose on validation data to minimize expected cost 10×FN + 1×FP, subject to clinical review capacity.

Evaluation	Stratified cross-validation for development; if dates are available, add a later-time holdout. Report recall, precision, PR-AUC, ROC-AUC, calibration and expected cost.
Explanation	Show the path/rules that triggered each alert, key feature values, uncertainty and a clinician review warning.

7.3 Cost-sensitive threshold and two illustrative confusion matrices

Expected cost = 10×FN + 1×FP. Compare two hypothetical thresholds on the same 1,000 patients (60 actual readmissions, 940 non-readmissions). These are worked examples to demonstrate the calculation, not experimental results.

Threshold	TP	FN	FP	TN	Cost = 10FN + FP	Recall
Higher threshold – fewer alerts	30	30	20	920	10(30)+20 = 320	30/60 = 50%
Lower threshold – more alerts	48	12	100	840	10(12)+100 = 220	48/60 = 80%

The lower threshold has more false positives but lower total cost (220 versus 320) because false negatives are ten times as expensive. The best threshold is not automatically the one with highest recall: calculate cost and confirm alert workload and clinical capacity.

7.4 Why accuracy is inappropriate

A model that predicts 'not readmitted' for every patient achieves 94% accuracy because 94% do not return within 30 days. However, it detects none of the high-risk patients (recall=0), which is unacceptable when missed cases are ten times more costly. Recall, precision, PR-AUC and expected cost are more aligned with the task.

7.5 Stage 4 – experimental validation plan

Use a public analogue dataset with a clearly defined readmission target. Document source, version, inclusion criteria, target coding and limitations. Run a majority-class baseline and the final cost-sensitive/interpretable model on the same folds. Compare no imbalance handling against class weighting or training-fold resampling. Never resample the test set. Report actual confusion matrices, recall, precision, ROC-AUC, PR-AUC and total cost. The supplied code package includes an experiment template; replace the placeholder dataset path/column names and run it.

Experiment	Recall	Precision	ROC-AUC	PR-AUC	Total cost	Evidence
Majority baseline	Fill after run	Fill after run	Fill after run	Fill after run	Fill after run	Confusion matrix
Pruned tree – unweighted	Fill after run	Fill after run	Fill after run	Fill after run	Fill after run	Tree and metrics
Pruned tree – cost-sensitive	Fill after run	Fill after run	Fill after run	Fill after run	Fill after run	Tree and metrics
Optional resampling	Fill after run	Fill after run	Fill after run	Fill after run	Fill after run	Training-only resampling result

7.6 Strengths, limitations and failure conditions

A shallow decision tree is understandable and can show why a patient was flagged, but may underfit complex clinical patterns and can be unstable when small data changes occur. Missing laboratory results may reflect clinical workflow rather than random absence. Diagnosis and treatment patterns may change over time, causing drift. The model can fail if follow-up data are incomplete, readmissions outside the network are not captured, or features leak information after discharge. Monitor calibration, subgroup performance, alert burden and drift; clinicians retain final responsibility.

7.7 Stage 5 – cross-defence and Stage 6 – joint evaluation

- Student A should explain which experimental evidence supports the design: recall, precision, PR-AUC, cost and confusion matrices on held-out data.

- Student B should explain how the 10:1 cost ratio is applied: total cost=10×false negatives+false positives, and why the threshold is chosen on validation data.
- The group should explain the trade-off: interpretability may sacrifice some predictive performance; lower thresholds may improve recall but increase false alerts.
- A data miner should inspect class balance, error costs, data volume, attribute types, missingness, feature timing, interpretability needs, fairness, deployment environment and evaluation design before choosing a classifier.

CONCLUSION

This assignment compares decision trees, Naïve Bayes, rule-based and associative classification, neural networks and support vector machines. The manual calculations show how information gain selects a tree split, how Bayes' theorem ranks classes, how backpropagation updates weights, how SVM margins are defined and how classification/prediction metrics are calculated. The evaluation and hospital design questions demonstrate that model choice must reflect class imbalance, error costs, interpretability, data quality and deployment constraints. Weka/Python outputs and screenshots must be added after running the experiments.

CONTRIBUTION RECORD – COMPLETE TRUTHFULLY

Question	Student A – primary contribution	Student B – primary contribution	Joint work
Q1	Manual induction / gains	Weka J48 validation	Agree criterion, compare trees
Q2	Bayes calculations	Weka and dependence experiment	Joint classifier choice
Q3	Rule coverage and conflicts	JRip/PART and CAR mining	Recommend representation
Q4	One-step backpropagation	MLP experiments	Interpret learning behaviour
Q5	SVM geometry	Kernel/scaling experiments	Choose suitable SVM setting
Q6	Metric calculations	Estimation methods / model comparison	Interpret meaningful evaluation
Q7	Analytical validation	Experimental validation	Independent proposals, design and cross-defence